

2D Rotation Note (Theory + Exact Matrix)

Relevant Files

- Main demo: `tutorials/lineDrawing/07_2d_rotation.py`
- Foundational guide: `tutorials/teaching_guides/04_triangle_rotation_guide.md`
- Composition demo: `tutorials/lineDrawing/10_composing_transformations.py`

Theoretical Base

Rotation preserves distances and angles while changing orientation around a center.

About origin:

- $x' = x\cos(\theta) - y\sin(\theta)$
- $y' = x\sin(\theta) + y\cos(\theta)$

About pivot $p = (p_x, p_y)$:

- $x' = \cos(\theta)(x - p_x) - \sin(\theta)(y - p_y) + p_x$
- $y' = \sin(\theta)(x - p_x) + \cos(\theta)(y - p_y) + p_y$

Exact Matrix Forms

About origin (2x2)

$$\begin{vmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{vmatrix}$$

About origin (3x3 homogeneous)

$$\begin{vmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{vmatrix}$$

About arbitrary pivot $p = (p_x, p_y)$

$$R_p = T(p_x, p_y) * R(\theta) * T(-p_x, -p_y)$$

Expanded exact 3x3 matrix:

$$\begin{vmatrix} \cos(\theta) & -\sin(\theta) & p_x - p_x \cos(\theta) + p_y \sin(\theta) \\ \sin(\theta) & \cos(\theta) & p_y - p_x \sin(\theta) - p_y \cos(\theta) \\ 0 & 0 & 1 \end{vmatrix}$$

Composition Rule (Non-Origin Workflow)

For a non-origin pivot:

1. Translate by $T(-p_x, -p_y)$
2. Rotate by $R(\theta)$
3. Translate back by $T(p_x, p_y)$

Parameter Meaning

- θ : rotation angle
- $p = (p_x, p_y)$: pivot

Convention:

- Positive θ is counterclockwise in Cartesian space.

Worked Example

Rotate $P = (3, 1)$ about origin by $\theta = 90$ deg:

- $\cos(90 \text{ deg}) = 0$
- $\sin(90 \text{ deg}) = 1$

Then:

- $x' = 3 \cdot 0 - 1 \cdot 1 = -1$
- $y' = 3 \cdot 1 + 1 \cdot 0 = 3$

Result: $P' = (-1, 3)$

How to Verify in Code

In `tutorials/lineDrawing/07_2d_rotation.py`:

- Set `angle = 0` -> transformed shape overlaps original.
- Set `angle = 180` -> each point maps to opposite side around pivot.
- Move pivot and confirm pivot remains fixed while others rotate.

Common Mistakes

- Using degrees directly in $\sin/\cos$ without converting to radians.
- Rotating already rotated coordinates repeatedly (drift accumulation).
- Forgetting order: translate to pivot first, then rotate, then translate back.

Quick Practice

1. Compare outputs for 90, 180, and 270 degrees.
2. Rotate around origin and around a clicked pivot; explain difference.
3. Verify $R(\theta) * R(-\theta)$ gives identity transform.